LAPORAN PRAKTIKUM

Naik Turun Kecepatan

Mata Kuliah ROBOTIKA

Nama Dino Rosalino Yuswanto

NPM 23081010152

Informatika Fakultas Ilmu Komputer

1. Ringkasan dan Target Pengujian

1.1 Ringkasan tugas

Praktikum dilakukan dengan menguji program arduino apakah mampu menaikkan dan menurunkan kecepatan motor melalui Python.

1.2 Lingkungan pengujian

Komponen	Versi / Spesifikasi
Bahasa / framework	Python & Arduino
Perangkat / simulator	ESP 32

1.3 Berkas yang diuji

Nama berkas	Peran / fungsi	Hubungan dengan berkas lain
On_off_motor.ino	Menggerakkan motor	Menerima data dari file on_off.ipynb
On_off.ipynb	Mengirimkan perintah untuk menggerakkan motor	Mengirimkan input yang diterima oleh on_off_motor.ino yang kemudian akan menyalakan atau mematikan motor

2. Kode Program

2.1 Potongan kode utama

Kode on_off_motor.ino

```
int ledPin = 2;
char myData = 0;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// Setting PWM properties
const int freq = 5000;          // diturunkan agar driver motor responsif
const int pwmChannel = 0;
const int resolution = 8;
int dutyCycle = 205;

const byte pin_rpm = 13;
volatile unsigned int pulseCount = 0;
unsigned long lastRpmTime = 0;
float rpm = 0.0;

// Jumlah slot/pulses per putaran (sesuaikan dengan encoder/sensor Anda)
// Contoh: 1 slot = 1 pulse per rotasi
const int PULSES_PER_REV = 1;

// ===== ISR: dipanggil setiap pulse dari sensor RPM =====
void IRAM_ATTR rpmISR() {
  pulseCount++;
}

void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);
```

```
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);
  pinMode(pin_rpm, INPUT_PULLUP);

  // Aktifkan interrupt pada pin RPM
  attachInterrupt(digitalPinToInterrupt(pin_rpm), rpmISR, RISING);

  ledcSetup(pwmChannel, freq, resolution);
  ledcAttachPin(enable1Pin, pwmChannel);

  Serial.println("Testing DC Motor...");
  lastRpmTime = millis();
}

void MotorOn()
{
  dutyCycle = 100;
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);
}

void MotorOff()
{
  dutyCycle = 0;
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);
}

// ===== Hitung RPM setiap 1 detik =====
void hitungRPM() {
  unsigned long now = millis();
  unsigned long elapsed = now - lastRpmTime;
```

```
  if (elapsed >= 1000) {  // setiap 1 detik
    noInterrupts();
    unsigned int count = pulseCount;
    pulseCount = 0;
    interrupts();

    rpm = (count * 60.0) / (PULSES_PER_REV * (elapsed / 1000.0));

    // Kirim ke Serial dalam format mudah dibaca Python: "RPM:xxx.xx"
    Serial.print("RPM:");
    Serial.println(rpm, 2);

    lastRpmTime = now;
  }
}

void loop()
{
  // Baca perintah dari Python
  if (Serial.available() > 0) {
    myData = Serial.read();

    if (myData == '1') {
      Serial.println("LED is on !!!");
      digitalWrite(ledPin, HIGH);
      MotorOn();
    }
    else if (myData == '0') {
      Serial.println("LED is off !!!");
      digitalWrite(ledPin, LOW);
      MotorOff();
    }
  }

  // Update RPM terus-menerus
```

```
  hitungRPM();
}
```

```
int ledPin = 2;
char myData = 0;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// Setting PWM properties
const int freq = 5000;          // diturunkan agar driver motor responsif
const int pwmChannel = 0;
const int resolution = 8;
int dutyCycle = 205;

const byte pin_rpm = 13;
volatile unsigned int pulseCount = 0;
unsigned long lastRpmTime = 0;
float rpm = 0.0;

// Jumlah slot/pulses per putaran (sesuaikan dengan encoder/sensor Anda)
// Contoh: 1 slot = 1 pulse per rotasi
const int PULSES_PER_REV = 1;

// ===== ISR: dipanggil setiap pulse dari sensor RPM =====
void IRAM_ATTR rpmISR() {
  pulseCount++;
}

void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);

  pinMode(motor1Pin1, OUTPUT);
```

```
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);
  pinMode(pin_rpm, INPUT_PULLUP);

  // Aktifkan interrupt pada pin RPM
  attachInterrupt(digitalPinToInterrupt(pin_rpm), rpmISR, RISING);

  ledcSetup(pwmChannel, freq, resolution);
  ledcAttachPin(enable1Pin, pwmChannel);

  Serial.println("Testing DC Motor...");
  lastRpmTime = millis();
}

void MotorOn()
{
  dutyCycle = 100;
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);
}

void MotorOff()
{
  dutyCycle = 0;
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);
}

// ===== Hitung RPM setiap 1 detik =====
void hitungRPM() {
  unsigned long now = millis();
  unsigned long elapsed = now - lastRpmTime;

  if (elapsed >= 1000) {  // setiap 1 detik
```

```
    noInterrupts();
    unsigned int count = pulseCount;
    pulseCount = 0;
    interrupts();

    rpm = (count * 60.0) / (PULSES_PER_REV * (elapsed / 1000.0));

    // Kirim ke Serial dalam format mudah dibaca Python: "RPM:xxx.xx"
    Serial.print("RPM:");
    Serial.println(rpm, 2);

    lastRpmTime = now;
  }
}

void loop()
{
  // Baca perintah dari Python
  if (Serial.available() > 0) {
    myData = Serial.read();

    if (myData == '1') {
      Serial.println("LED is on !!!");
      digitalWrite(ledPin, HIGH);
      MotorOn();
    }
    else if (myData == '0') {
      Serial.println("LED is off !!!");
      digitalWrite(ledPin, LOW);
      MotorOff();
    }
  }

  // Update RPM terus-menerus
  hitungRPM();
}
```

Kode on_off.ipynb

```
# ============================================================
# Program Kontrol Motor ESP32 via Serial (Auto-Detect Port)
# + Menampilkan Kecepatan Motor (RPM)
# ============================================================
# Kirim '1' untuk memutar motor, '0' untuk mematikan motor,
# 'r' untuk membaca RPM dari ESP32
# ============================================================

import serial
import serial.tools.list_ports
import time
import sys
import re

# ==========================================
# KONFIGURASI
# ==========================================
BAUDRATE = 115200  # Harus sama dengan Serial.begin() di ESP32

# Kata kunci pada deskripsi port yang menandakan ESP32 / USB-UART
ESP32_KEYWORDS = [
    "cp210", "cp2102", "cp210x",       # Silicon Labs (umum di ESP32 DevKit)
    "ch340", "ch341", "ch34x",         # WCH (umum di ESP32 clone)
    "ftdi", "ft232",                   # FTDI
    "usb-serial", "usb serial",
    "usb uart", "uart bridge",
    "silicon labs",
    "espressif", "esp32",
]

# (VID, PID) yang umum untuk USB-to-UART ESP32
ESP32_VID_PID = {
    (0x10C4, 0xEA60),  # Silicon Labs CP210x
```

```
    (0x10C4, 0xEA70),  # Silicon Labs CP2105
    (0x1A86, 0x7523),  # WCH CH340
    (0x1A86, 0x55D4),  # WCH CH9102
    (0x0403, 0x6001),  # FTDI FT232
    (0x0403, 0x6015),  # FTDI FT231X
    (0x303A, 0x1001),  # Espressif USB JTAG/Serial
}


# ============================================
# Fungsi untuk melihat daftar port yang tersedia
# ============================================
def list_serial_ports():
    """Menampilkan semua port serial yang terdeteksi."""
    ports = serial.tools.list_ports.comports()
    if not ports:
        print("Tidak ada port serial yang terdeteksi.")
        return []
    print("Port serial yang tersedia:")
    for p in ports:
        vid_pid = ""
        if p.vid is not None and p.pid is not None:
            vid_pid = f" [VID:PID={p.vid:04X}:{p.pid:04X}]"
        print(f"  - {p.device} : {p.description}{vid_pid}")
    return list(ports)


# ============================================
# Fungsi auto-detect port ESP32
# ============================================
def deteksi_port_esp32(verbose=True):
    """
    Mendeteksi port serial ESP32 secara otomatis.
    Return: nama port (str) atau None jika tidak ditemukan.
    """
    ports = serial.tools.list_ports.comports()
    if not ports:
        if verbose:
```

```
            print("[!] Tidak ada port serial yang terdeteksi.")
        return None

    kandidat = []

    for p in ports:
        desc = (p.description or "").lower()
        mfg = (p.manufacturer or "").lower()
        hwid = (p.hwid or "").lower()
        teks = f"{desc} {mfg} {hwid}"

        skor = 0

        # 1) Cocokkan VID:PID
        if p.vid is not None and p.pid is not None:
            if (p.vid, p.pid) in ESP32_VID_PID:
                skor += 100

        # 2) Cocokkan kata kunci deskripsi/manufacturer
        for kw in ESP32_KEYWORDS:
            if kw in teks:
                skor += 50
                break

        # 3) Hindari port Bluetooth (sering muncul di Windows)
        if "bluetooth" in teks:
            skor -= 200

        if skor > 0:
            kandidat.append((skor, p))

    if not kandidat:
        # Fallback: kalau cuma ada 1 port non-bluetooth, pakai itu
        non_bt = [p for p in ports
                  if "bluetooth" not in (p.description or "").lower()]
        if len(non_bt) == 1:
            if verbose:
```

```
                print(f"[i] Tidak ada ciri ESP32 yang cocok, "
                      f"tapi hanya ada 1 port non-Bluetooth: {non_bt[0].device}")
            return non_bt[0].device

        if verbose:
            print("[!] Port ESP32 tidak dapat dideteksi otomatis.")
            print("    Silakan tentukan PORT secara manual.")
        return None

    # Urutkan berdasarkan skor tertinggi
    kandidat.sort(key=lambda x: x[0], reverse=True)

    if verbose:
        print("Kandidat port ESP32 (skor tertinggi = paling mungkin):")
        for skor, p in kandidat:
            print(f"  [{skor:>4}] {p.device} : {p.description}")

    return kandidat[0][1].device


# ============================================
# Membuka koneksi serial
# ============================================
def buka_koneksi(port, baudrate=115200):
    """Membuka koneksi serial ke ESP32."""
    try:
        ser = serial.Serial(port, baudrate, timeout=1)
        time.sleep(2)  # Tunggu ESP32 reset setelah koneksi dibuka
        print(f"Terhubung ke ESP32 di {port} @ {baudrate} baud")
        return ser
    except serial.SerialException as e:
        print(f"Gagal membuka port {port}: {e}")
        return None


# ============================================
# Fungsi kirim perintah ke ESP32
# ============================================
```

```python
def kirim_perintah(ser, perintah: str):
    """
    Mengirim perintah ke ESP32.
    perintah: '1' untuk ON, '0' untuk OFF
    """
    if perintah not in ('0', '1'):
        raise ValueError("Perintah harus '0' atau '1'")

    ser.write(perintah.encode('utf-8'))
    ser.flush()
    time.sleep(0.1)

    # Baca balasan dari ESP32 (opsional)
    if ser.in_waiting > 0:
        balasan = ser.readline().decode('utf-8', errors='ignore').strip()
        if balasan:
            print(f"ESP32: {balasan}")


def motor_on(ser):
    """Menyalakan motor (kirim '1')."""
    print(">> Mengirim '1' : Motor ON")
    kirim_perintah(ser, '1')


def motor_off(ser):
    """Mematikan motor (kirim '0')."""
    print(">> Mengirim '0' : Motor OFF")
    kirim_perintah(ser, '0')


# ============================================
# Fungsi baca RPM dari ESP32
# ============================================
def baca_rpm(ser, timeout=1.5, tampilkan_semua=False):
    """
    Membaca nilai RPM dari ESP32.
    ESP32 mengirim dalam format: 'RPM:123.45'
    Return: float RPM terakhir atau None jika tidak ada data.
```

```
    """
    waktu_mulai = time.time()
    rpm_terakhir = None

    while time.time() - waktu_mulai < timeout:
        if ser.in_waiting > 0:
            try:
                baris = ser.readline().decode('utf-8', errors='ignore').strip()
            except Exception:
                continue

            if not baris:
                continue

            # Cari baris dengan format RPM:xxx
            match = re.search(r"RPM:([\d\.]+)", baris)
            if match:
                rpm_terakhir = float(match.group(1))
                print(f"   >> Kecepatan motor: {rpm_terakhir:.2f} RPM")
            elif tampilkan_semua:
                # Tampilkan pesan lain dari ESP32 (mis. "LED is on !!!")
                print(f"   ESP32: {baris}")
        else:
            time.sleep(0.05)

    if rpm_terakhir is None:
        print("   [!] Tidak ada data RPM yang diterima dari ESP32.")
    return rpm_terakhir


def pantau_rpm(ser, durasi=5):
    """
    Menampilkan RPM secara terus-menerus selama `durasi` detik.
    """
    print(f"--- Memantau RPM selama {durasi} detik ---")
    waktu_mulai = time.time()
    rpm_terakhir = None
    while time.time() - waktu_mulai < durasi:
```

```
        rpm = baca_rpm(ser, timeout=1.2)
        if rpm is not None:
            rpm_terakhir = rpm
    print(f"--- Selesai memantau. RPM terakhir: "
          f"{rpm_terakhir if rpm_terakhir is not None else 'N/A'} ---")
    return rpm_terakhir


# ============================================
# PROGRAM UTAMA
# ============================================
if __name__ == "__main__":

    # 1) Tampilkan port yang tersedia
    print("=" * 50)
    list_serial_ports()
    print("=" * 50)

    # 2) Auto-detect port ESP32
    PORT = deteksi_port_esp32()
    if PORT is None:
        print("Program dihentikan: port ESP32 tidak ditemukan.")
        sys.exit(1)

    print(f"\n>> Port ESP32 terpilih: {PORT}")

    # 3) Buka koneksi serial
    esp32 = buka_koneksi(PORT, BAUDRATE)

    if esp32 is None:
        print("Koneksi gagal. Periksa kembali PORT dan kabel USB.")
    else:
        try:
            # 4) Contoh urutan kontrol motor + tampilkan RPM
            print("\n--- Mulai urutan kontrol motor ---")

            motor_on(esp32)
            pantau_rpm(esp32, durasi=3)   # tampilkan RPM selama motor ON
```

```
            motor_off(esp32)
            time.sleep(1)

            motor_on(esp32)
            pantau_rpm(esp32, durasi=3)

            motor_off(esp32)
            print("--- Urutan kontrol selesai ---\n")

            # 5) Mode interaktif
            print("Mode interaktif:")
            print("  '1' = Motor ON")
            print("  '0' = Motor OFF")
            print("  'r' = Baca RPM sekali")
            print("  'p' = Pantau RPM (5 detik)")
            print("  'q' = Keluar")
            while True:
                cmd = input("Perintah: ").strip().lower()

                if cmd == 'q':
                    break
                elif cmd in ('0', '1'):
                    kirim_perintah(esp32, cmd)
                elif cmd == 'r':
                    baca_rpm(esp32, timeout=2)
                elif cmd == 'p':
                    pantau_rpm(esp32, durasi=5)
                else:
                    print("Perintah tidak valid! Gunakan '1', '0', 'r', 'p', atau
'q'.")

        except KeyboardInterrupt:
            print("\nDihentikan oleh pengguna.")

        finally:
            # 6) Pastikan motor mati & tutup koneksi
```

```
            try:
                motor_off(esp32)
            except Exception:
                pass
            esp32.close()
            print("Koneksi serial ditutup.")
}
```

3. Penjelasan Kode

3.1 Penjelasan fungsi / blok

Fungsi / blok kode	Cara kerja	Input / output atau efek
Konfigurasi Python	Menetapkan BAUDRATE 115200, kata kunci USB-UART, dan daftar VID/PID untuk memberi skor pada port yang mungkin digunakan ESP32.	Input: daftar port sistem. Output: kandidat port dengan skor tertinggi.
list_serial_ports()	Mengambil port yang terdeteksi melalui pyserial dan menampilkan nama, deskripsi, serta VID/PID jika tersedia.	Output: informasi port COM yang tersedia.
deteksi_port_esp32()	Menilai port berdasarkan VID/PID dan kata kunci deskripsi. Port Bluetooth diberi pengurangan skor. Kandidat dengan skor tertinggi dipilih.	Output: nama port, misalnya COM6, atau None jika tidak ditemukan.
buka_koneksi()	Membuka port serial dengan baud rate 115200 dan timeout 1 detik, lalu menunggu 2 detik agar ESP32 selesai reset.	Output: objek serial jika berhasil; None jika gagal.
kirim_perintah(), motor_on(), motor_off()	Mengirim karakter '1' atau '0' melalui serial. Fungsi memvalidasi perintah dan melakukan flush setelah pengiriman.	'1' menyalakan motor; '0' mematikan motor.
baca_rpm()	Membaca baris serial sampai batas waktu, mencari pola RPM:angka dengan regular expression, lalu mengubah angka menjadi float.	Output: nilai RPM terakhir atau None jika tidak ada data RPM.
pantau_rpm()	Memanggil baca_rpm() berulang selama durasi yang ditentukan dan menampilkan RPM terakhir yang diterima.	Output: pemantauan RPM selama beberapa detik.
setup() Arduino	Mengatur pin LED, pin arah motor, pin PWM, dan pin sensor RPM. Memulai serial pada 115200, mengaktifkan interrupt sensor, serta mengatur PWM.	Efek: ESP32 siap menerima perintah dan menghitung pulsa sensor.
MotorOn() / MotorOff()	MotorOn mengatur arah motor dan PWM ke 100. MotorOff mengatur PWM ke 0. Keduanya mempertahankan kombinasi pin arah HIGH/LOW.	Efek: sinyal kendali driver motor berubah.
rpmISR() dan hitungRPM()	Interrupt menambah pulseCount setiap tepi naik pada pin RPM. hitungRPM menghitung RPM setiap sekitar 1 detik, lalu mengirim baris RPM:xxx.xx.	Input: pulsa sensor pada GPIO 13. Output: data RPM ke serial.
loop() Arduino	Memeriksa data serial yang tersedia. Karakter '1' menyalakan LED dan motor; '0' mematikan LED dan motor. Fungsi hitungRPM tetap dipanggil.	Menerima perintah dan memperbarui keluaran motor serta data RPM.

3.2 Alur eksekusi

Alur Python :

1. Mengimpor pustaka `serial`, `serial.tools.list_ports`, `time`, dan `sys`, lalu menetapkan baud rate 115200 serta parameter pendeteksian port.
2. Menampilkan daftar port yang tersedia dan menghitung skor kandidat berdasarkan VID/PID serta kata kunci perangkat USB-UART.

3. Memilih port dengan skor tertinggi. Jika port tidak ditemukan, program dihentikan; jika ditemukan, Python mencoba membuka koneksi serial.
4. Menjalankan demo otomatis: motor ON, menaikkan duty cycle tiga kali, menurunkan duty cycle dua kali, menetapkan PWM 150, kemudian motor OFF.
5. Masuk ke mode interaktif. Pengguna dapat mengetik `1`, `0`, `+`, `-`, `p<angka>`, atau `q` untuk mengirim perintah maupun keluar.
6. Ketika program dihentikan, Python mencoba mengirim perintah OFF, menutup koneksi serial, dan menampilkan pesan penutupan.

Alur Arduino :

1. Fungsi `setup()` menyiapkan komunikasi serial 115200 baud, pin motor dan LED, serta konfigurasi PWM.
2. ESP32 mengawali keluaran PWM pada 0 agar motor berada dalam keadaan mati, kemudian mengirim pesan bahwa pengendali siap.
3. Fungsi `loop()` membaca karakter yang diterima dan mengumpulkannya sampai penanda akhir baris diterima.
4. Jika perintah berupa `PWM:<angka>`, angka diproses oleh `setDuty()`. Jika perintah berupa `1`, `0`, `+`, atau `-`, fungsi motor yang sesuai dijalankan.
5. Nilai duty cycle diterapkan ke kanal PWM. LED menyala bila duty cycle lebih besar dari 0, lalu ESP32 mengirim informasi status melalui Serial.

4. Hasil Eksekusi dan Pengujian

FOTO HASIL 1

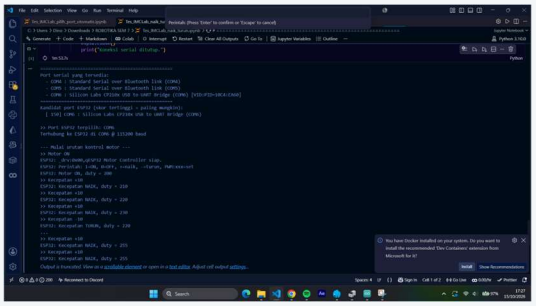
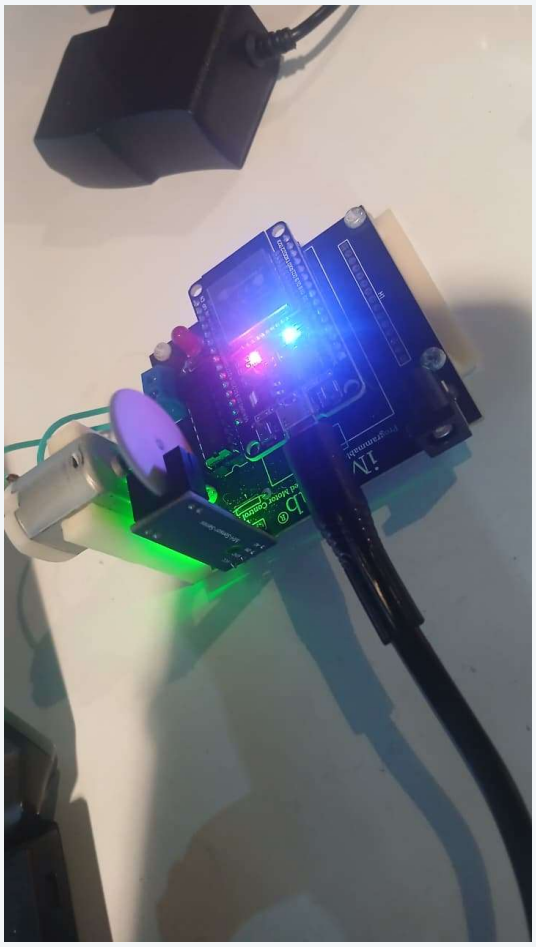


FOTO HASIL 2



5. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, motor DC dapat dikendalikan menggunakan bahasa pemrograman Python melalui komunikasi serial dengan ESP32. Pengendalian motor tidak hanya mencakup fungsi menyalakan dan mematikan motor, tetapi juga pengaturan kecepatan melalui perubahan nilai *duty cycle* Pulse Width Modulation (PWM). Perintah yang dikirimkan melalui Python meliputi `'1'` untuk menyalakan motor, `'0'` untuk mematikan motor, `'+'` untuk menaikkan nilai PWM, `'-'` untuk menurunkan nilai PWM, serta `PWM:xxx` untuk menentukan nilai PWM secara langsung pada rentang 0–255. ESP32 kemudian memproses perintah tersebut dan mengatur keluaran PWM melalui driver motor, dengan LED sebagai indikator kondisi keluaran motor. Praktikum ini menunjukkan bahwa integrasi Python dan ESP32 melalui komunikasi serial dapat digunakan untuk menerapkan pengendalian motor secara lebih fleksibel. Dengan demikian, praktikum ini memberikan pemahaman mengenai penerapan PWM dalam pengaturan motor DC, meskipun perubahan nilai PWM tidak selalu menghasilkan perubahan kecepatan yang linier karena dipengaruhi oleh karakteristik motor, beban, dan catu daya yang digunakan.